Who submitted the workload?

Reproduction steps, observed metrics and current gaps | 16 October 2026

Exporter configuration

The PoC adds `submitted_by` to the existing Pod and workload metrics. Admission records direct Kubernetes callers. CEL reads platform metadata from workloads and Pods.

```
exporter:
  enabled: true
  namespace: karta-demo
  useCatalog: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "workload.dgxc.nvidia.com/user",
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

`exporter.namespace: karta-demo` limits workload and Pod LIST/WATCH requests, including health checks, to that namespace. For each platform below, select the namespace containing its workload Pods and update the query filter. For a comparison across namespaces, use `namespace: ""` and remove the query filter. Karta definitions and required cluster-scoped resources remain cluster-wide.

Admission scope is separate: it selects requests that receive the caller annotation. Platform workloads can use their existing metadata without admission. Keep the operator and webhook enabled for direct capture; set `admission.enabled: false` for extraction only.

An empty `submitted_by` means unknown. In Prometheus or Grafana Explore:

```
karta_workload_status{namespace="karta-demo"} == 1
karta_pod_workload_info{namespace="karta-demo"}
```

The first query shows each workload's current phase and submitter. The second shows its Pods. The examples below show selected metric labels.

Direct Kubernetes: choose the scope and protect the caller

Merge these settings into the exporter values. Native `matchConditions` requires Kubernetes 1.30+. The same conditions apply to capture and update protection.

```
exporter:
  attribution:
    admission:
      enabled: true
      namespaceSelector:
        matchExpressions:
        - key: karta.run.ai/capture-submitter
          operator: In
          values: ["true"]
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values: [kube-system, karta-system]
      resourceRules:
      - apiGroups: ["batch"]
        apiVersions: ["v1"]
        resources: ["jobs", "cronjobs"]
      matchConditions:
      - name: exclude-cronjobs
        expression: request.kind.kind != "CronJob"
```

The administrator creates `karta-demo` and labels it `karta.run.ai/capture-submitter=true`. Give the test user create/get/patch access to Jobs there. Replace `karta-system` with the operator namespace. This example captures Jobs and excludes CronJobs. Use stable resource conditions so later updates stay protected.

With the user's kubeconfig:

```
cat > direct-job.yaml <<'YAML'
apiVersion: batch/v1
kind: Job
metadata: {name: submitter-job, namespace: karta-demo}
spec:
  backoffLimit: 0
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      automountServiceAccountToken: false
      restartPolicy: Never
      containers:
      - name: worker
        image: busybox:1.37.0
        command: [sh, -c, "echo submitter-demo; sleep 180"]
YAML
kubectl --kubeconfig "$USER_KUBECONFIG" apply -f direct-job.yaml
KEY=workload.dgxc.nvidia.com/submitter-id
kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-demo \
  get job submitter-job -o json | jq '.metadata.annotations'
```

Observed: the Job annotation contained the caller, and the unlabeled Pod inherited it in both Karta metrics. The finite Job completed.

```
# Changing the recorded user is rejected.
kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-demo \
  annotate job submitter-job "$KEY=other@example.test" --overwrite
# Claiming another user at creation is also rejected.
kubectl create --dry-run=client -f direct-job.yaml -o json |
  jq --arg key "$KEY" '.metadata.name="forged" |
    .metadata.annotations[$key]="other@example.test"' |
  kubectl --kubeconfig "$USER_KUBECONFIG" create -f -
```

Both attempts were rejected. Namespace inclusion/exclusion and the CronJob condition were also exercised in the cluster. Protection applies inside the configured scope; a service account identifies that service, not a separate human.

Run:AI

Set `RUNAI_API`, `RUNAI_TOKEN`, `PROJECT_ID` and `CLUSTER_ID` using your normal login and project. Submit training and inference:

```
jq -n --arg p "$PROJECT_ID" --arg c "$CLUSTER_ID" '{
  name: "cel-label-training", projectId: $p, clusterId: $c,
  spec: {
    image: "busybox:1.37.0", command: "sh",
    args: "-c \"echo attribution-start; sleep 300; echo done\"",
    backoffLimit: 0, restartPolicy: "Never",
    compute: {
      cpuCoreRequest: 0.05, cpuCoreLimit: 0.2,
      cpuMemoryRequest: "64Mi", cpuMemoryLimit: "128Mi",
      gpuDevicesRequest: 0
    }
  }
}' > training.json

jq '
  .name="cel-label-inference" |
  del(.spec.backoffLimit, .spec.restartPolicy) |
  .spec.args=("-c \"mkdir -p /tmp/www; " +
    "echo attribution-inference > /tmp/www/index.html; " +
    "httpd -f -p 8080 -h /tmp/www\"") |
  .spec.autoscaling={minReplicas:1,maxReplicas:1} |
  .spec.servingPort={
    container:8080,protocol:"http",clusterLocalAccessOnly:true}
' training.json > inference.json

for kind in training inference; do
  curl -fsS "$RUNAI_API/api/v1/workloads/${kind}s" \
    -H "Authorization: Bearer $RUNAI_TOKEN" \
    -H 'Content-Type: application/json' --data-binary @"$kind.json"
done
```

Observed: training completed; inference served HTTP 200. Run:AI placed the same identity on the Job and Knative Service:

```
# Job/cel-label-training and Service/cel-label-inference
metadata:
  annotations:
    run.ai/submitted-by: test@run.ai
```

The WorkloadRunner had no submitter key. CEL read the annotation on the Job/Service; the inference Pod metric inherited the Service's value.

```
karta_workload_status{workload="cel-label-training",
  phase="Completed", submitted_by="test@run.ai"} 1
karta_pod_workload_info{workload="cel-label-inference",
  submitted_by="test@run.ai"} 1
```

Setup: the exporter needs get/list/watch on `run.ai/workloadrunners` plus its standard Job, Pod, apps and Knative resources. Add a ClusterRole labeled `karta.run.ai/exporter-rbac: "true"`. This inference path also required Knative's `kubernetes.podspec-schedulername` and `kubernetes.podspec-affinity` flags.

Current gap: agree the NVIDIA username format with Run:AI. Submitter propagation worked in both paths; no external identity mapper was used.

OSMO

Use the logged-in OSMO CLI. Replace `YOUR_CPU_PLATFORM` and `YOUR_POOL`, then submit this leader/worker group:

```
cat > osmo-group.yaml <<'YAML'
workflow:
  name: cel-label-group
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  groups:
  - name: pair
    tasks:
    - name: leader
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
      lead: true
    - name: worker
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-group.yaml --pool YOUR_POOL
osmo workflow query YOUR_WORKFLOW_ID
```

Observed: the API returned `submitted_by: alice@example.test`. Task Pods and the KAI PodGroup contained:

```
metadata:
  labels:
    osmo.submitted_by: alice
```

CEL reads that existing label. Both the single-task and leader/worker workflows completed. The Karta metrics contained `submitted_by="alice"`.

```
karta_pod_workload_info{workload_kind="Pod", submitted_by="alice"} 1
karta_workload_status{workload_kind="Pod", phase="Running", submitted_by="alice"} 1
```

To also publish the agreed common label, register a named Pod template:

```
podTemplates:
  karta-submitter:
    metadata:
      labels:
        workload.dgxc.nvidia.com/user: "{{WF_SUBMITTED_BY | default('')}}"
```

Place this under `services.configs` in 6.3.1, or `configuration` in 6.4 (inside `configuration.snapshot` when used). Append `karta-submitter` to the selected platform's `override_pod_template` list, preserving its existing names.

Current gaps: OSMO shortens email identities and omits the one-character user `a`. The guard keeps missing attribution empty. A maintained administrator map from a unique short username to a full principal worked; it cannot recover ambiguous usernames. Prefer exposing the full authenticated principal in a trusted annotation. OSMO's task metrics already carry the full stored user for reporting joins.

The live tests used development-authentication headers. Production IdP identity mapping still needs verification. Karta currently reports OSMO's bare task Pods individually; integrating their existing workflow/group IDs is a separate Karta gap.

NVCF: container function

Set `NVCF_API` / `NVCF_ADMIN_TOKEN` from your installation and choose an available `GPU` / `INSTANCE_TYPE`. Management permissions: `register_function`, `deploy_function`, `delete_function`, and `list_functions` or `list_functions_details`. API-key permissions use the `apikey:` prefix.

```
curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @- \
  > function.json <<'JSON'
{
  "name":"cel-label-function",
  "containerImage":"nvcr.io/nvidia/omniverse/simple-nginx:1.0.2",
  "inferenceUrl":"/", "inferencePort":80,
  "health":{"protocol":"HTTP","uri":"/","port":80,
            "timeout":"PT10S","expectedStatusCode":200}
}
JSON
FID=$(jq -r '.function.id' function.json)
VID=$(jq -r '.function.versionId' function.json)
DEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$FID/versions/$VID"
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  '{deploymentSpecifications:[{gpu:$gpu,instanceType:$it,
    minInstances:1,maxInstances:1,maxRequestConcurrency:1}]}' \
  > deploy.json
curl -fsS "$DEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @deploy.json
curl -fsS "$NVCF_API/v2/nvcf/functions/$FID/versions/$VID" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

Observed: ACTIVE and HTTP 200. Pods carried account, function/version and ICMSRequest IDs, but no authenticated submitter annotation. Both Karta metric families kept an empty identity:

```
karta_pod_workload_info{workload_kind="Pod", submitted_by=""} 1
karta_workload_status{workload_kind="Pod", phase="Running", submitted_by=""} 1
```

Supplying `KARTA_TEST_SUBMITTED_BY` through `containerEnvironment` reached the Pod spec, but did not become metadata. CEL reads labels and annotations, not container environment variables.

Release the allocation before the next test:

```
curl -fsS -X DELETE "$DEPLOY" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

Current gap: native metadata identifies the account/workload, not the authenticated creator or deployer. Existing audit events contain actor/subject identity and correlation IDs. A log pipeline can join them for reporting. For Karta's metadata path, prefer server-controlled creator/deployer/submitter annotations; shared credentials remain service identities.

Lab: NVCF 1.18.0, NVCT 1.66.0, NVCA 3.10.0, ReVal 0.20.2. The HTTP/function and task fixtures used fake GPU allocation. The two native-container test Pods needed a lab-only CPU resize to 100m, separate from the API requests shown here.

NVCT: task that completes

Set `NVCT_API` / `NVCT_API_KEY` with `launch_task` and `task_details`. The task writes the required progress state, reaches 100, then exits:

```
cat > nvct-task.sh <<'SH'
set -eu
for attempt in $(seq 1 60); do
  [ -d "$NVCT_RESULTS_DIR" ] && break
  sleep 1
done
[ -d "$NVCT_RESULTS_DIR" ]
sleep 1
progress() {
  printf '{"taskId":"%s","percentComplete":%s,"name":"%s","lastUpdatedAt":"%s"}\n' \
    "$NVCT_TASK_ID" "$1" "$2" "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
    > "$NVCT_PROGRESS_FILE_PATH.tmp"
  mv "$NVCT_PROGRESS_FILE_PATH.tmp" "$NVCT_PROGRESS_FILE_PATH"
}
progress 1 running
sleep 180
progress 100 completed
sleep 10
SH
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  --arg script "$(base64 < nvct-task.sh | tr -d '\n')" '{
  name:"cel-label-task", gpuSpecification:{gpu:$gpu,instanceType:$it},
  containerImage:"nvcr.io/nvidia/cuda:12.6.3-base-ubuntu22.04",
  containerArgs:"bash -c '\''printf %s \"$ATTRIBUTION_SCRIPT\" | base64 -d | bash'\''",
  maxRuntimeDuration:"PT5M", maxQueuedDuration:"PT10M",
  terminationGracePeriodDuration:"PT15S", resultHandlingStrategy:"NONE",
  containerEnvironment:[{key:"ATTRIBUTION_SCRIPT",value:$script}]
}' > task.json
curl -fsS "$NVCT_API/v1/nvct/tasks" \
  -H "Authorization: Bearer $NVCT_API_KEY" \
  -H 'Content-Type: application/json' --data-binary @task.json > task-result.json
TASK_ID=$(jq -r '.task.id' task-result.json)
curl -fsS "$NVCT_API/v1/nvct/tasks/$TASK_ID" \
  -H "Authorization: Bearer $NVCT_API_KEY"
```

Observed: COMPLETED at 100%. A supplied environment value reached the task process, but the Pod still had no submitter metadata. Both Karta metrics kept `submitted_by=""`, as in the function test.

The lab required these helper-image settings in the existing deployment values:

```
nvctApi:
  remoteConfig:
    enabled: true
    configData:
      nvct:
        sidecars:
          init-container: "nvcr.io/nvidia/nvcf/nvcf-worker-init-oss:1.2.1"
          utils-container: "nvcr.io/nvidia/nvcf/nvcf-worker-task-oss:1.1.4"
```

Current gap: retain and publish the authenticated task submitter. Earlier malformed-task cancellation left an ICMSRequest for manual cleanup. Treat helper-image packaging and failed-request cleanup separately from identity propagation; reproduce those on the target release before opening bugs.

NVCF Helm: existing metadata path

The Helm configuration can already pass a supplied identity to the Pod. The NIM chart below ran a pinned Python HTTP fixture. Save these values:

```
cat > helm-values.json <<'JSON'
{
  "fullnameOverride":"karta-helm-attribution-retest",
  "statefulSet":{"enabled":false}, "replicaCount":1,
  "image":{
    "repository":"docker.io/library/python",
    "tag":"3.12-alpine@sha256:1b668429b3511ab407d8e00648891631b0b1a4d7e15e3ca70f38ab5b91ad4ab4",
    "pullPolicy":"IfNotPresent"
  },
  "customCommand":["python3","-m","http.server","8080","--directory","/tmp"],
  "env":[{"name":"PYTHONDONTWRITEBYTECODE","value":"1"}],
  "imagePullSecrets":[{"name":"karta-http-fixture-unused-pull"}],
  "model":{"ngcAPIKey":"unused-http-fixture",
           "ngcAPISecret":"karta-http-fixture-placeholder","openaiPort":8080},
  "podAnnotations":{
    "workload.dgxc.nvidia.com/submitter-id":"transport-test-only"
  },
  "podSecurityContext":{"runAsUser":1001,"runAsGroup":1001,"fsGroup":1001},
  "containerSecurityContext":{"allowPrivilegeEscalation":false,
    "capabilities":{"drop":["ALL"]},"runAsNonRoot":true},
  "resources":{"requests":{"cpu":"50m","memory":"64Mi"},
    "limits":{"cpu":"250m","memory":"128Mi","nvidia.com/gpu":1}},
  "service":{"type":"ClusterIP","name":"karta-helm-attribution-retest",
             "openaiPort":8080},
  "startupProbe":{"path":"/","initialDelaySeconds":0},
  "readinessProbe":{"path":"/","initialDelaySeconds":0},
  "livenessProbe":{"path":"/"},
  "serviceAccount":{"create":false}, "persistence":{"enabled":false},
  "multiNode":{"enabled":false}, "ingress":{"enabled":false}
}
JSON
```

`unused-http-fixture` is a placeholder, not a credential. This HTTP fixture does not download a model. NVCF used its existing NGC credential to pull the chart.

Required lab settings:

- On NVCA 3.10.0, ReVal used Kubernetes v1.20.0 by default. Set the operator chart's `k8sVersionOverride` to the actual server version; the lab used `v1.35.5`.
- The lab lacked SMB CSI. For this storage-free function, append `-HelmSharedStorage` to the NVCA operator chart's existing `worker.featureFlags`. It affects the backend; preserve other flags and restore it afterward. Helm tasks/shared-storage functions need their normal storage setup.
- The existing ReVal setting was `reval.serviceConfig.skipSanitizeObjectMetadata: true`. With sanitization enabled, `preserveAnnotations` controls exact root keys. This chart stamps the Pod template; stricter-mode behavior is outside this live result.

NVCF Helm: submission and result

Using the management token and available GPU/instance type from the function example:

```
CHART=https://helm.ngc.nvidia.com/nvidia/charts/nim-llm-1.14.0-pb5.0.tgz
jq -n --arg chart "$CHART" '{
  name:"karta-helm-attribution-retest-20261016", helmChart:$chart,
  helmChartServiceName:"karta-helm-attribution-retest",
  inferenceUrl:"/", inferencePort:8080, healthUri:"/", apiBodyFormat:"CUSTOM"
}' | curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @- > helm-function.json
HFID=$(jq -r '.function.id' helm-function.json)
HVID=$(jq -r '.function.versionId' helm-function.json)
HDEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$HFID/versions/$HVID"
HFUNCTION="$NVCF_API/v2/nvcf/functions/$HFID/versions/$HVID"
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  --slurpfile values helm-values.json \
  '{deploymentSpecifications:[{gpu:$gpu,instanceType:$it,
    minInstances:1,maxInstances:1,maxRequestConcurrency:1,
    configuration:$values[0]}]}' |
  curl -fsS "$HDEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
    -H 'Content-Type: application/json' --data-binary @-
curl -fsS "$HFUNCTION" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" |
  jq '.function.status'
```

Observed: ACTIVE; the function Service returned HTTP 200 through a port-forward. The Pod contained:

```
metadata:
  annotations:
    workload.dgxc.nvidia.com/submitter-id: transport-test-only
```

The Deployment root had no submitter annotation. Karta followed Pod -> ReplicaSet -> Deployment and used the Pod identity for both metrics. The first page's CEL already reads this key:

```
karta_pod_workload_info{workload="karta-helm-attribution-retest",
  workload_kind="Deployment", submitted_by="transport-test-only"} 1
karta_workload_status{workload="karta-helm-attribution-retest",
  phase="Running", submitted_by="transport-test-only"} 1
```

Workload observation health was 1. This proves metadata transport. `transport-test-only` was caller-supplied; it does not prove who authenticated. An existing trusted submission service could derive and protect the value, or NVCF could capture and propagate it natively. That authenticated capture remains the feature request. The separate Helm gateway attempt used a management token and returned 403.

After capture:

```
curl -fsS -X DELETE "$HDEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS -X DELETE "$HFUNCTION" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

The lab test was removed and temporary settings restored. Test evidence, OSMO request, NVCF request.